21CSC202J - OPERATING SYSTEMS

LAB MANUAL



II YEAR / III Semester

Academic Year: 2025-2026 ODD SEMESTER

FACULTY OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

RAMAPURAM CHENNAI- 600 089

LIST OF EXPERIMENTS

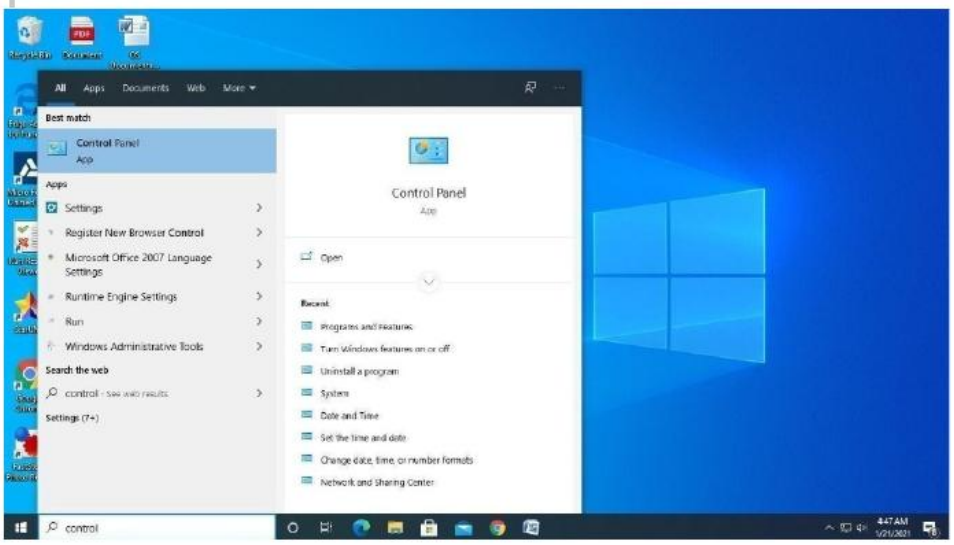
Ex.No.	Experiment Name
1	A. Operating system Installation, B. Basic Linux commands, C.Filter and admin commands
2	Process Creation using fork() and Usage of getpid(), getppid(), wait() functions
3	Multithreading
4	Mutual Exclusion using semaphore and monitor
5	Reader-Writer problem
6	Dining Philosopher problem
7	Bankers Algorithm for Deadlock avoidance
8	FCFS and SJF Scheduling
9	Priority and Round robin scheduling
10	FIFO Page Replacement Algorithm
11	LRU and LFU Page Replacement Algorithm

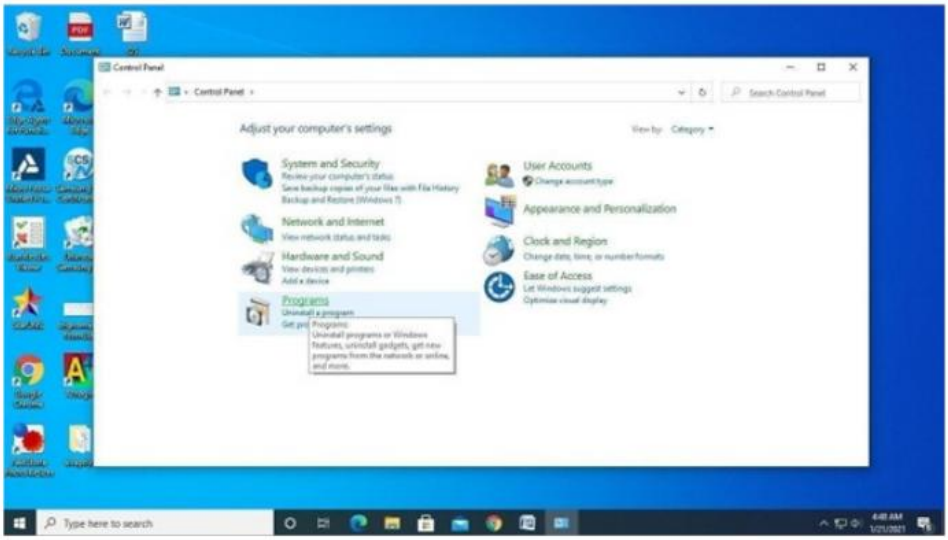
12	Best fit and Worst fit memory management policies
13	Disk Scheduling algorithm
14	Sequential and Indexed file Allocation
15	File organization schemes for single level and two level directory

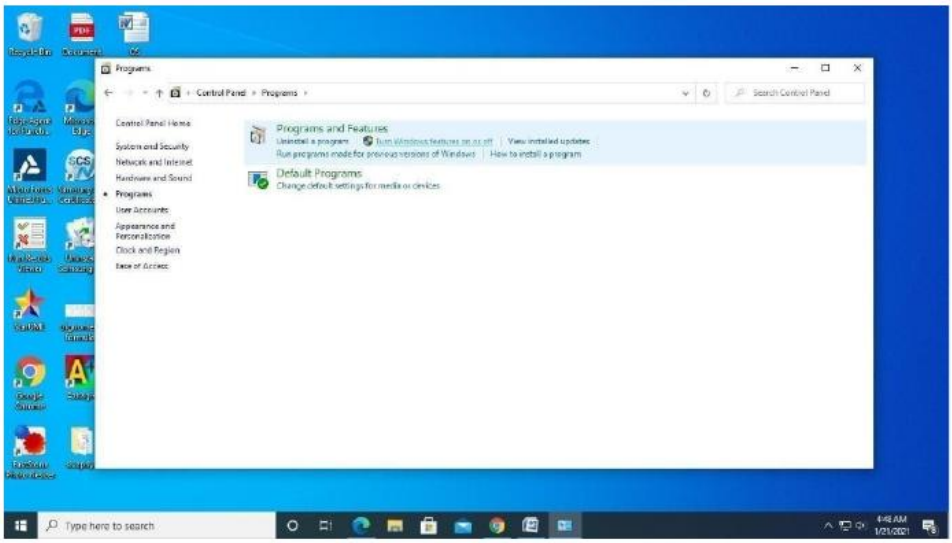
EX.NO.1A	**Operating System Installation**

Ubuntu installation guidelines in windows

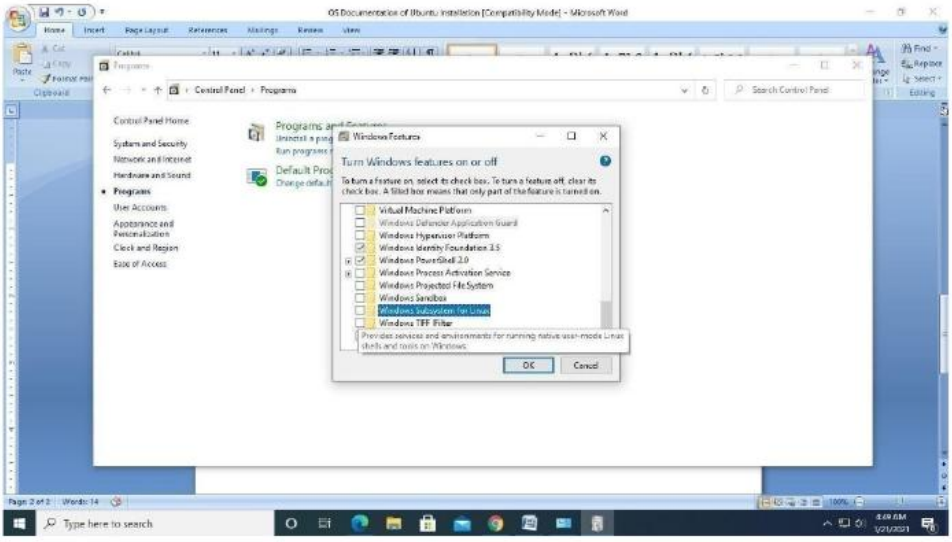
Installing Ubuntu in windows 10- 64 bit



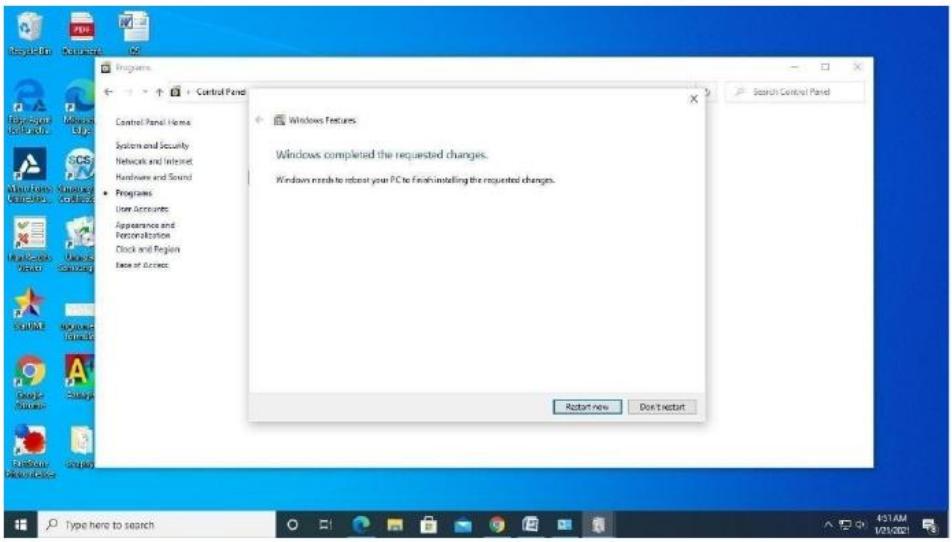




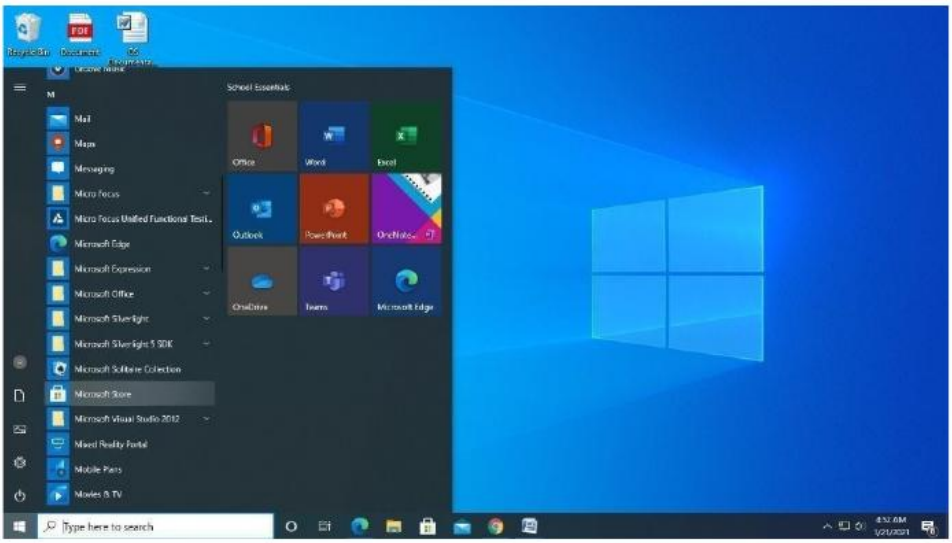
Select Turn Windows features On or Off



Select Windows subsystem for linux then press Ok

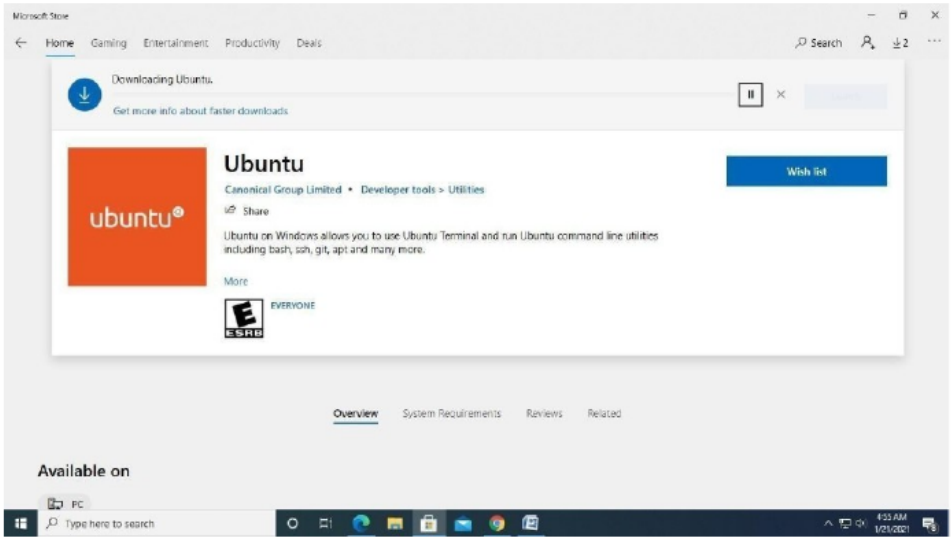


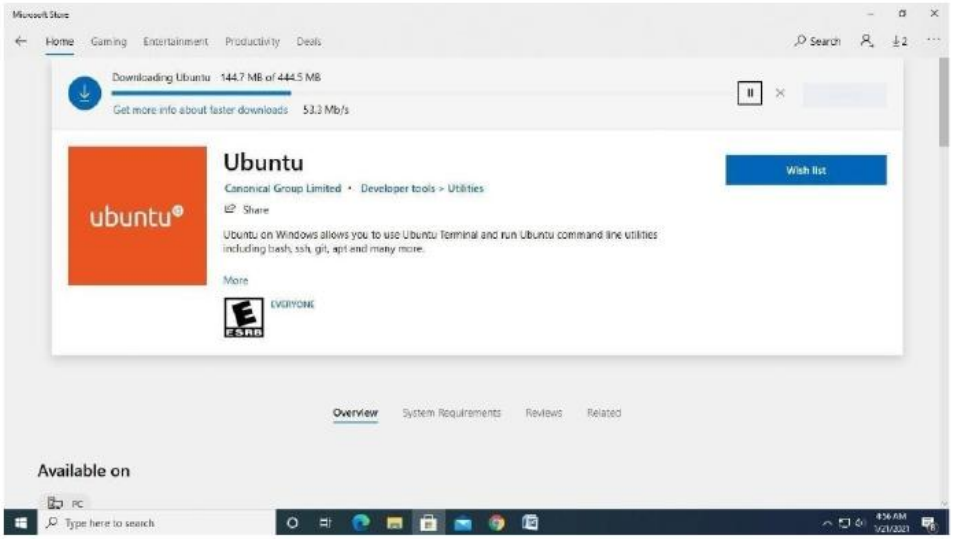
Now restart the PC to apply the changes

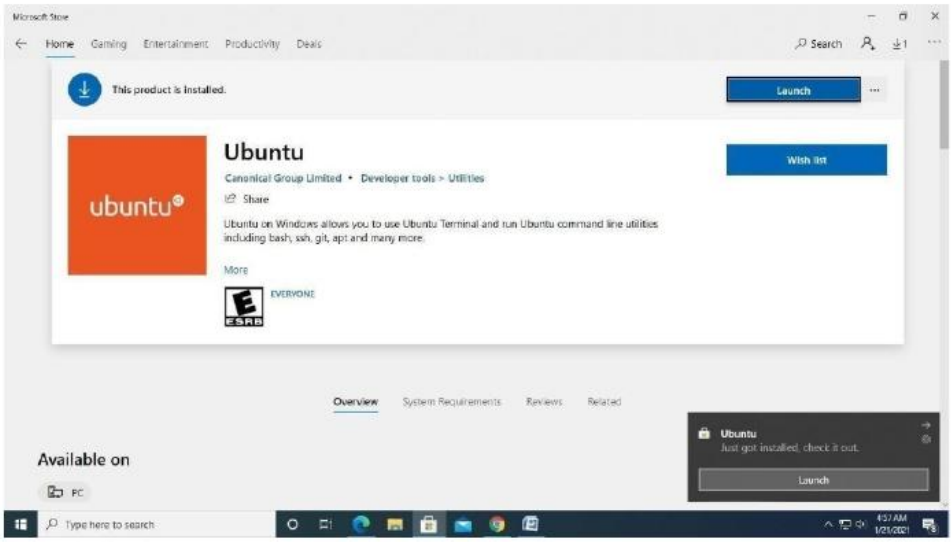


Choose Microsoft Store and search for Ubuntu and Install it

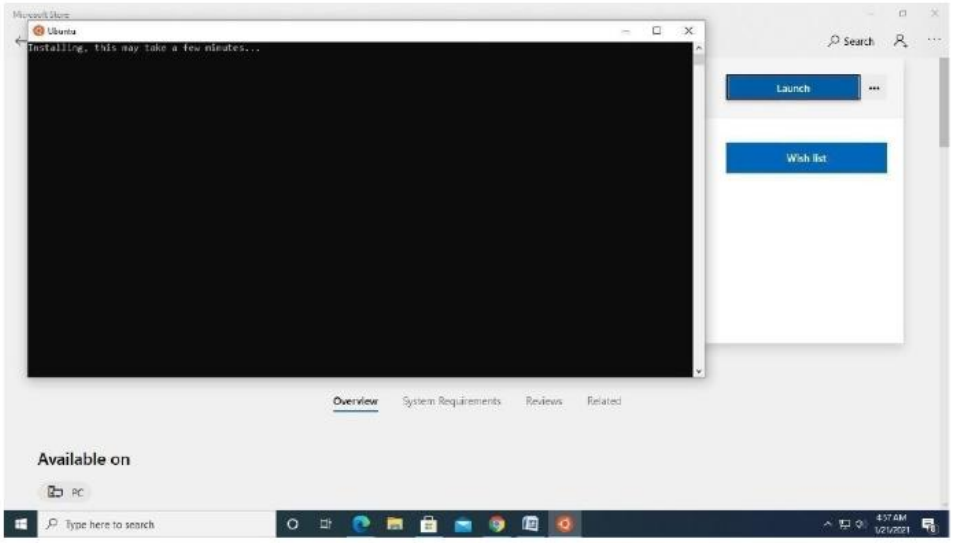
Click on Install

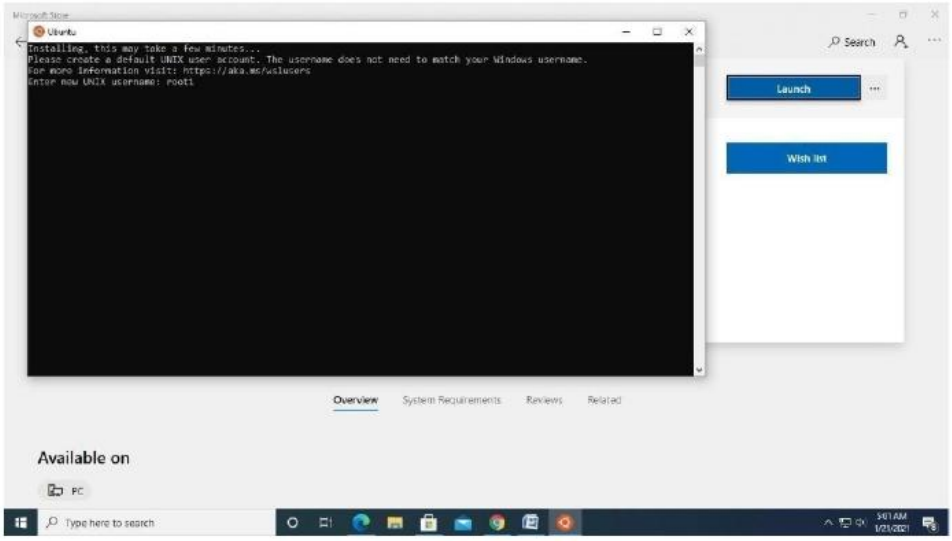






Now Launch the Ubuntu





Username : Any meaningful name
Password: any name

EXPERIMENT - 3

TITLE - MediCore: Intelligent Healthcare Operations and Patient Monitoring

AIM : To write a program for single-processor multi-threading and multi-processor multi-threading

Definitions -

Thread

A thread represents the foundational unit of CPU execution, consisting of a distinct program counter, register state, and stack memory, while sharing common code, data, and system resources with other threads belonging to the same process.

Threading

Threading is running multiple parts of a program (or seemingly at the same time), so it doesn't have to wait for one task to finish before starting another.

Process

A process is an independent running instance of a program, with its own separate memory and resources.

Multi-processing

Multiprocessing is an operating system and hardware computing architecture in which two or more autonomous central processing units (CPUs or cores) reside within a single computer system, sharing primary physical memory, system buses, and peripheral I/O devices, to achieve true simultaneous execution (hardware parallelism) of multiple independent instruction streams or processes under coordinated system control.

Part A. (Single-process multithreading)

Algorithm

Step 1 : Start.

Step 2 :Define `N = 4` and declare the `reading` array.

Step 3 :Input HR, SpO2, Temperature, and BP readings.

Step 4:Create four threads, one for each device.

Step 5:Pass the device ID to each thread.

Step 6:Each thread reads and displays its corresponding device reading.

Step 7:Wait for all threads to complete using `pthread_join()`.

Step 8:Stop.

Program

```
#include <stdio.h>
#include <pthread.h>
#define N 4
int reading[N];
void* device(void* arg) {
   int id = *(int*)arg;
   printf("Device %d reading: %d\n", id, reading[id]);
}
int main() {
   pthread_t t[N]; int id[N];
   printf("Enter HR, SpO2, Temp, BP: ");
   for (int i=0;i<N;i++) scanf("%d",&reading[i]);
   for (int i=0;i<N;i++){ id[i]=i; pthread_create(&t[i],NULL,device,&id[i]); }
   for (int i=0;i<N;i++) pthread_join(t[i],NULL);
   return 0;
}
```

Part B[multi-process multi-threading]

Algorithm

Step 1: Start.

Step 2: Declare an integer array `val[2]` to store two device readings.

Step 3: Read two device readings from the user and store them in `val`.

Step 4: Create two child processes using `fork()`.

Step 5: In each child process, create two threads using `pthread_create()`.

Step 6: Assign device IDs `0` and `1` to the threads.

Step 7: Each thread accesses its corresponding reading and displays the process ID, device ID, and reading.

Step 8: Wait for both threads to complete using `pthread_join()`.

Step 9: Terminate each child process using `_exit()`.

Step 10: In the parent process, wait for both child processes using `wait()`.

Step 11: Stop.

Program

```
#include <stdio.h>

#include <pthread.h>

#include <unistd.h>

#include <sys/wait.h>

int val[2];

void* dev(void* a){ int i=*(int*)a; printf("PID %d Device %d reading %d\n", getpid(), i, val[i]); }

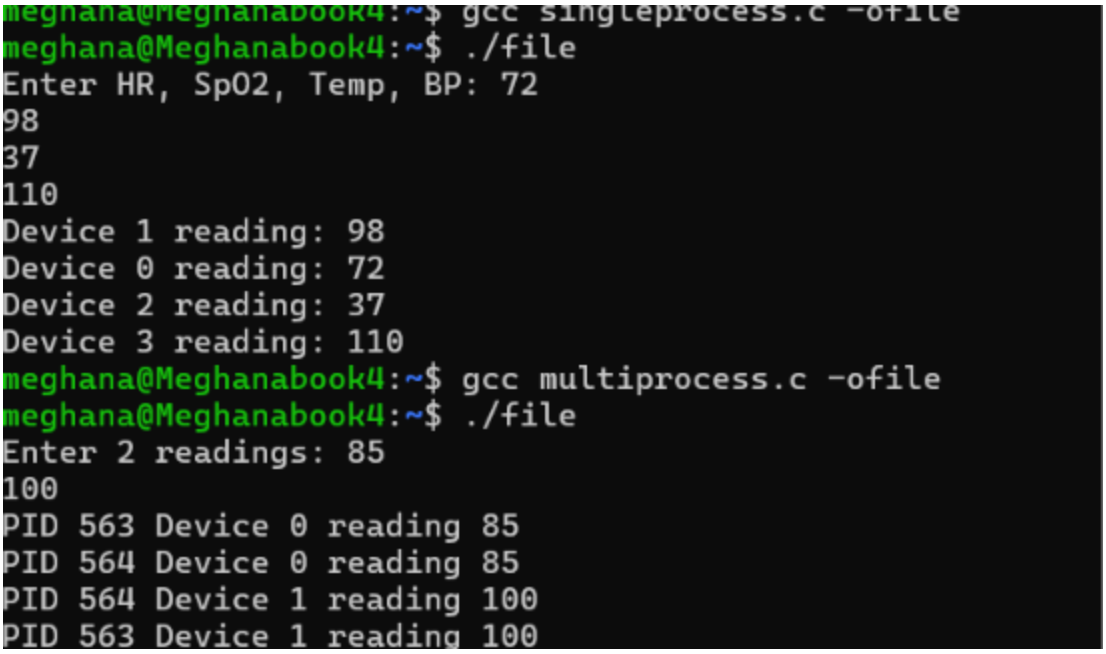
int main() {

    printf("Enter 2 readings: ");
```

```
    for (int i=0;i<2;i++) scanf("%d",&val[i]);
    for (int s=0; s<2; s++) {
        if (fork()==0) {
            pthread_t t[2]; int id[2]={0,1};
            for(int i=0;i<2;i++) pthread_create(&t[i],NULL,dev,&id[i]);
            for(int i=0;i<2;i++) pthread_join(t[i],NULL);
            _exit(0);
        }
    }
    for (int s=0; s<2; s++) wait(NULL);
    return 0;
}
```

Domain: Healthcare Information Systems
Use Case: Real-Time Patient Vital Monitoring

Output :



EXPERIMENT - 4

TITLE - AeroFlow: Real-Time Air Traffic Control and Coordination

AIM : To implement mutual exclusion using a semaphore and a monitor

Definitions -

Critical section

A critical section is a segment of code within a concurrent program that accesses and manipulates shared resources—such as memory locations, global variables, files, or hardware devices—which must not be executed concurrently by more than one process or thread to prevent race conditions and maintain data consistency.

Mutual exclusion.

It is a concurrency control property that ensures only one process or thread can access a shared resource (the critical section)At a time , preventing race conditions when multiple processes run together.

Semaphore

It is an integer Variable used for process synchronization, controlled by two atomic operations - wait() and signal() to lock a critical section.

Monitor.

A monitor is a high-level synchronization construct encapsulated as an abstract data type (ADT) that binds shared data variables, an initialization sequence, and a set of programmer-defined operations under implicit mutual exclusion, ensuring that only one active process or thread can execute inside the monitor at any given moment.

Part A [Semaphore basic solution]

Algorithm

Step 1: Start.

Step 2: Declare thread array `t[3]`, aircraft IDs `id[3] = {1, 2, 3}`, runway count `r`, and global semaphore `runway`.

Step 3: Input the number of available runways `r`.

Step 4: Initialize the semaphore `runway` with the value `r` using `sem_init()`.

Step 5: Create 3 concurrent threads using `pthread_create()`, executing the `aircraft` routine with their respective IDs.

Step 6: Inside each `aircraft` thread:

- Call `sem_wait(&runway)` to acquire an available runway (blocks if count is 0).
- Execute the critical section: print landing and clearance messages.
- Call `sem_post(&runway)` to release the runway and signal waiting threads.

Step 7: Wait for all threads to finish execution using `pthread_join()`.

Step 8: Stop.

Program

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
sem_t runway;
void* aircraft(void* id) {
    int n = *(int*)id;
    sem_wait(&runway);
    printf("Aircraft %d: Landing on runway...\n", n);
    printf("Aircraft %d: Runway cleared.\n", n);
    sem_post(&runway);
}
int main() {
    pthread_t t[3]; int id[3]={1,2,3}; int r;
    printf("Enter number of runways available: ");
    scanf("%d",&r);
    sem_init(&runway, 0, r);
    for(int i=0;i<3;i++) pthread_create(&t[i],0,aircraft,&id[i]);
    for(int i=0;i<3;i++) pthread_join(t[i],0);
}
```

Part B[monitor equivalent]

Algorithm

Algorithm

Step 1: Start.

Step 2: Statically initialize a global mutex lock runway using PTHREAD_MUTEX_INITIALIZER.

Step 3: Prompt for and read the number of aircraft n.

Step 4: Declare thread array t[n] and identifier array id[n].

Step 5: Loop i from *0* to $n - 1$:

- Assign id[i] = i + 1.
- Spawn thread t[i] using pthread_create() to execute the aircraft routine with argument &id[i].

Step 6: Inside the aircraft thread function:

- Dereference argument pointer to get aircraft ID n.
- Acquire the lock using pthread_mutex_lock(&runway) (blocks if another aircraft is currently using the runway).
- Execute critical section: print landing and clearance messages.
- Release the lock using pthread_mutex_unlock(&runway).

Step 7: Loop i from *0* to $n - 1$ and call pthread_join(t[i], NULL) to wait for all threads to terminate.

Step 8: Stop.

Program

```
#include <stdio.h>

#include <pthread.h>

pthread_mutex_t runway = PTHREAD_MUTEX_INITIALIZER;
```

```
void* aircraft(void* id) {
    int n = *(int*)id;
    pthread_mutex_lock(&runway);
    printf("Aircraft %d: Landing on runway...\n", n);
    printf("Aircraft %d: Runway cleared.\n", n);
    pthread_mutex_unlock(&runway);
}
int main() {
    int n;
    printf("Enter number of aircrafts: ");
    scanf("%d",&n);
    pthread_t t[n]; int id[n];
    for(int i=0;i<n;i++){ id[i]=i+1; pthread_create(&t[i],0,aircraft,&id[i]); }
    for(int i=0;i<n;i++) pthread_join(t[i],0);
}
```

Domain: Avionics and Intelligent Transportation Systems (ITS)

Use Case: Automated Airport Surface Traffic Control & Runway Access Arbitration

Output :

```
meghana@Meghanabook4:~$ gcc semaphoreA.c -ofile
meghana@Meghanabook4:~$ ./file
Enter number of runways available: 1
Aircraft 1: Landing on runway...
Aircraft 1: Runway cleared.
Aircraft 2: Landing on runway...
Aircraft 2: Runway cleared.
Aircraft 3: Landing on runway...
Aircraft 3: Runway cleared.
meghana@Meghanabook4:~$ gcc semaphoreB.c -ofile
meghana@Meghanabook4:~$ ./file
Enter number of aircrafts: 3
Aircraft 1: Landing on runway...
Aircraft 1: Runway cleared.
Aircraft 2: Landing on runway...
Aircraft 2: Runway cleared.
Aircraft 3: Landing on runway...
Aircraft 3: Runway cleared.
```

EXPERIMENT - 5

TITLE - READER AND WRITER PROBLEM

AIM : To study the reader-writer problem

Definition

Reader

A process that only reads data from a shared resource (like a file or database) multiple readers can read at the same time.

Writer

A process that modifies or writes data to the shared resources: only one writer should access it at a time and it Should not write while readers are reading .

Reader-Writer Problem

The Readers-Writers Problem is a classical computer science concurrency and synchronization benchmark that models access arbitration to a shared data repository (such as a database, shared memory segment, or file system cache)

among two competing categories of execution threads—readers, which inspect shared state without modification, and writers, which perform destructive updates—such that concurrent read operations are permitted without serialization, while write operations are granted strict, non-preemptive mutual exclusion over both concurrent writers and readers to preserve data consistency and prevent race conditions.

Algorithm

Step 1: Start.

Step 2: Declare global log message array logs[5] containing predefined event descriptions and an integer array flag[5] to hold status values.

Step 3: Inside main(), declare thread array t[5] and index array id[5] = {0, 1, 2, 3, 4}.

Step 4: Read 5 status flags (1 for threat, 0 for safe) from the user into flag[0] through flag[4].

Step 5: Loop i from *0* to *4*:

- Create concurrent thread t[i] using pthread_create(), passing the detect routine and argument &id[i].

Step 6: Inside the detect thread routine:

- Dereference the passed argument to obtain log index i.
- Check if flag[i] == 1:
    - If true, print "Threat <i>: <logs[i]> - ALERT!".
    - If false, print "Log <i>: <logs[i]> - Safe".
- Terminate thread execution and return NULL.

Step 7: Loop i from *0* to *4* and call pthread_join(t[i], NULL) to wait for all log inspection threads to finish.

Step 8: Stop.

Program

#include <stdio.h>

```
#include <pthread.h>

char *logs[]={"Normal Login","Failed Login","SQL Injection","Normal Access","Malware Detected"};

int flag[5];

void *detect(void *arg){

    int i=*(int*)arg;

    if(flag[i])

        printf("Threat %d: %s - ALERT!\n",i,logs[i]);

    else

        printf("Log %d: %s - Safe\n",i,logs[i]);

    return NULL;

}

int main(){

    pthread_t t[5]; int id[5]={0,1,2,3,4};

    printf("Enter threat flag for 5 logs (1=threat,0=safe): ");

    for(int i=0;i<5;i++) scanf("%d",&flag[i]);

    for(int i=0;i<5;i++) pthread_create(&t[i],NULL,detect,&id[i]);

    for(int i=0;i<5;i++) pthread_join(t[i],NULL);

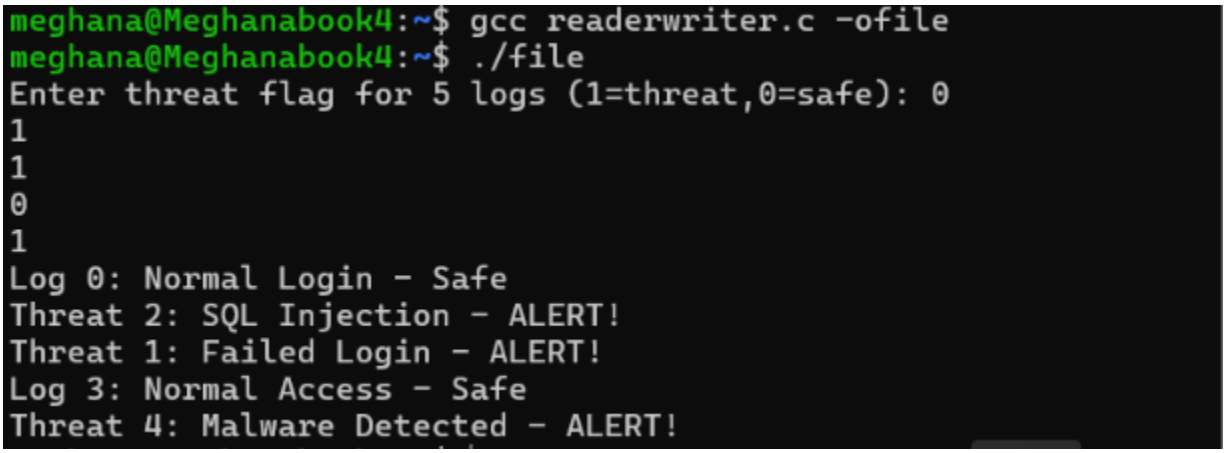
    return 0;

}
```

Domain: Cyber Security & Information Assurance

Use Case: Concurrent Security Event Log Triage and Real-Time Threat Classification

Output :



EXPERIMENT - 6

TITLE - Dining philosopher Problem.

Aim : To implement and study the dining philosophers' problem

Definitions.

Dining philosopher problem

The dining philosophers' problem is a classic synchronization problem in operating systems. It imagines five philosophers sitting around a table with five chopsticks and five bowls of rice to eat. A philosopher needs two chopsticks, one from their left and one from their right.

User of dining philosophers

Resource Allocation Modeling: It abstractly simulates modern OS scenarios where multiple worker threads or kernel processes concurrently request sets of non-preemptive hardware/software devices (e.g., dual-channel memory, multiple tape drives, or compound database locks).
Testing Deadlock Prevention Algorithms: It is widely used to demonstrate and validate structural solutions against cyclic wait conditions, such as:

- Resource hierarchies (enforcing strict numerical order when claiming locks).

- Asymmetric resource acquisition (e.g., odd-numbered philosophers acquire the left fork first, while even-numbered acquire the right fork first).
- State inspection via monitors or semaphores (allowing a thread to acquire resources only when both are simultaneously available).

Disadvantages of dining philosophers
Deadlock: A circular wait occurs if all philosophers grab their left fork simultaneously, freezing the system forever.
Livelock: Threads may repeatedly yield and re-grab forks in sync, burning CPU time without making any real progress.
Starvation: Alternating neighbors can repeatedly share forks, leaving the philosopher between them waiting indefinitely.
Low Concurrency: At most two philosophers can eat at once, leaving idle resources underutilized.
Unrealistic Topology: Real OS resources rarely follow a simple, symmetrical circle.

Domain: Healthcare Automation and Clinical Robotics

Use Case: Hospital Bedside Medical Equipment Sharing and Conflict Resolution

Algorithm

Step 1: Start.

Step 2: Declare an array of 5 mutex locks tool[5] representing shared medical devices, thread array t[5], and robot identifier array id[5] = {0, 1, 2, 3, 4}.

Step 3: Prompt the user to enter the number of active bedside robots n (≤ 5).

Step 4: Initialize the first n mutexes using pthread_mutex_init(&tool[i], NULL).

Step 5: Loop i from *0* to $n - 1$:

- Create concurrent thread t[i] using pthread_create(), passing routine robot and argument &id[i].

Step 6: Inside the robot thread routine:

- Dereference argument pointer to get robot index i.
- Acquire the primary tool lock: pthread_mutex_lock(&tool[i]).
- Acquire the adjacent tool lock: pthread_mutex_lock(&tool[(i + 1) % 5]).
- Execute the critical section: print "Robot <i>: Task done using tool <i> and <(i+1)%5>".
- Release both locks: call pthread_mutex_unlock(&tool[i]) and pthread_mutex_unlock(&tool[(i + 1) % 5]).
- Terminate thread execution and return NULL.

Step 7: Loop i from *0* to $n - 1$ and call pthread_join(t[i], NULL) to wait for all robot threads to finish.

Step 8: Stop.

Program.

```
#include <stdio.h>

#include <pthread.h>

pthread_mutex_t tool[5];

void* robot(void* a){ int i=*(int*)a;

   pthread_mutex_lock(&tool[i]); pthread_mutex_lock(&tool[(i+1)%5]);

   printf("Robot %d: Task done using tool %d and %d\n",i,i,(i+1)%5);

   pthread_mutex_unlock(&tool[i]); pthread_mutex_unlock(&tool[(i+1)%5]);

   return NULL; }

int main(){ int n; pthread_t t[5]; int id[5]={0,1,2,3,4};

   printf("Enter number of robots (max 5): "); scanf("%d",&n);

   for(int i=0;i<n;i++) pthread_mutex_init(&tool[i],NULL);

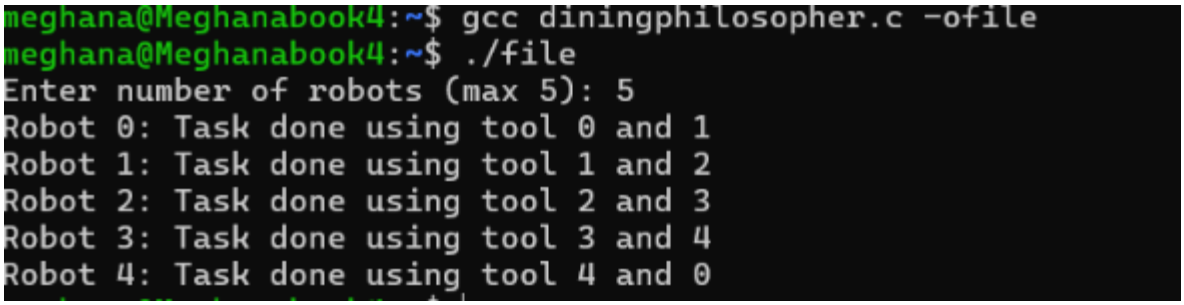
   for(int i=0;i<n;i++) pthread_create(&t[i],NULL,robot,&id[i]);
```

```
    for(int i=0;i<n;i++) pthread_join(t[i],NULL);

    return 0;

}
```

Output.



EXPERIMENT - 7

TITLE - Banker's algorithm and deadlock avoidance

Aim : To implement and study banker's algorithm for the deadlock avoidance problem

Definitions.

The Banker's Algorithm is a deadlock avoidance algorithm developed by Edsger Dijkstra that tests for safety by simulating the allocation of predetermined maximum possible amounts of all resources, and then making an "s-state" check to test for possible activities, before deciding whether allocation should be allowed to continue.

Uses

1. **Deadlock Avoidance in Operating Systems:** Evaluates dynamic resource requests from multiple processes in real time to ensure the system never enters an unsafe state that could lead to deadlock.

2. **Distributed & Multi-Resource Banking Systems:** Manages concurrent transactions and credit/cash limits across accounts to prevent total liquidity freeze or circular wait conditions.

How It Works

1. **Request Validation:** When a process requests resources, the system checks if $Request \leq Need$. If valid, it checks if $Request \leq Available$; otherwise, the process must wait.
2. **Pretend Allocation:** The OS temporarily pretends to grant the request by modifying the state vectors:
    - $Available = Available - Request$
    - $Allocation = Allocation + Request$
    - $Need = Need - Request$
3. **Safety Algorithm Execution:** The system searches for a safe sequence by finding an unfinished process whose remaining $Need \leq Available$. When found, it assumes that process completes, reclaims its allocated resources ($Available = Available + Allocation$), and repeats for all processes.
4. **Final Decision:** If a complete safe sequence exists where every process can finish, the state is declared **Safe** and the actual allocation is granted. If no safe sequence exists, the allocation is rolled back and the process is forced to wait.

Domain: Cloud Infrastructure and Virtualization Management

Use Case: Deadlock-Free Dynamic CPU Core Allocation in Virtual Machine Clusters

Algorithm

Step 1: Start.

Step 2: Declare integer variables n (number of VMs), avail (available CPU cores), cnt = 0 (completed VM counter), and arrays alloc[10], need[10], and fin[10] initialized to all zeros.

Step 3: Prompt for and read the total number of VMs n and the initial available CPU cores avail.

Step 4: For each VM i from *0* to $n - 1$:

- Prompt for and read its currently allocated cores alloc[i] and remaining required cores need[i].

Step 5: While completed VM count $cnt < n$:

- Initialize flag p = -1.
- Search for an unallocated VM: Iterate i from *0* to $n - 1$:
  - If $fin[i] == 0$ and $need[i] \leq avail$, set $p = i$ and break the loop.
- If $p == -1$ (no eligible VM found), print "System UNSAFE" and terminate execution.
- If an eligible VM p is found:
  - Reclaim its allocated resources: $avail = avail + alloc[p]$.
  - Mark it complete: set $fin[p] = 1$.
  - Increment completion counter: $cnt = cnt + 1$.
  - Print execution trace: "VM <p> runs, Available=<avail>".

Step 6: If the loop completes with all VMs finished ($cnt == n$), print "System is in SAFE state".

Step 7: Stop.

Program.

```
#include <stdio.h>

int main(){

   int n,avail,alloc[10],need[10],fin[10]={0},cnt=0;

   printf("Enter number of VMs, available CPU cores: "); scanf("%d
%d",&n,&avail);

   for(int i=0;i<n;i++){ printf("Enter alloc,need for VM %d: ",i); scanf("%d
%d",&alloc[i],&need[i]); }

   while(cnt<n){ int p=-1;

      for(int i=0;i<n;i++) if(!fin[i]&&need[i]<=avail){ p=i; break; }

      if(p==-1){ printf("System UNSAFE\n"); return 0; }
```

```
        avail+=alloc[p]; fin[p]=1; cnt++; printf("VM %d runs,
Available=%d\n",p,avail);

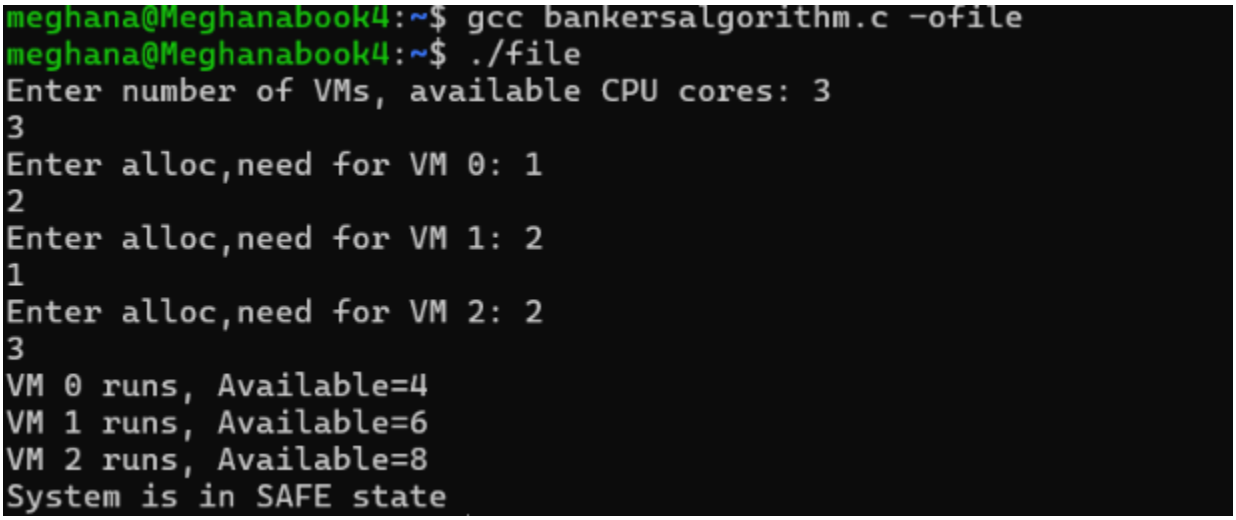
    }

    printf("System is in SAFE state\n");

    return 0;

}
```

Output



EXPERIMENT - 8

TITLE - FCFS and SJF scheduling.

Aim : To study the concepts of FCFS and SJF scheduling

Definition.

First-come, first-served (FCFS).

First-Come, First-Served (FCFS) is a fundamental, non-preemptive CPU scheduling policy wherein the central processing unit is allocated to processes strictly in the chronological order of their arrival in the ready queue, managed through a standard First-In, First-Out (FIFO) queue data structure, such that the process that requests the CPU first retains execution control until it voluntarily terminates or initiates an input/output operation.

Shortest Job First (SJF)

Shortest-Job-First (SJF) is an optimal CPU scheduling algorithm that associates with each process the length of its next CPU burst cycle, dispatching the processor to the process with the smallest predicted execution time, operating in either a non-preemptive mode where an assigned process runs to completion, or a preemptive mode (known as Shortest Remaining Time First, or SRTF) where the arrival of a process with a shorter remaining burst displaces the currently executing process.

Domain: Industrial Automation and Smart Manufacturing

Use Case: Real-Time Embedded CPU Task Scheduling for Automated Assembly Lines

First-come, first-served (FCFS) :

Algorithm

Step 1: Start.

Step 2: Declare integer variables n (number of processes), t = 0 (current cumulative time/completion time tracker), and arrays bt[10] (burst time) and wt[10] (waiting time).

Step 3: Prompt for and read the total number of processes n.

Step 4: For each process i from *0* to $n - 1$:

- Prompt for and read the burst time into bt[i].

Step 5: For each process i from *0* to $n - 1$:

- Assign the current cumulative time to waiting time: wt[i] = t.
- Update the cumulative completion time: t = t + bt[i].
- Print execution trace: "P<i>: Waiting=<wt[i]>, Completion=<t>".

Step 6: Stop.

Program

```
#include <stdio.h>

int main(){

   int n,bt[10],wt[10],t=0;

   printf("Enter number of processes: "); scanf("%d",&n);

   for(int i=0;i<n;i++){ printf("Enter burst time of P%d: ",i); scanf("%d",&bt[i]); }

   for(int i=0;i<n;i++){ wt[i]=t; t+=bt[i]; printf("P%d: Waiting=%d,
Completion=%d\n",i,wt[i],t); }

   return 0;

}
```

Shortest Job First (SJF) :

Algorithm

Step 1: Start.

Step 2: Declare integer variables n (number of processes), t = 0 (cumulative timeline/completion tracker), and arrays bt[10] (burst time), id[10] (process identifiers), and wt[10] (waiting time).

Step 3: Prompt for and read the total number of processes n.

Step 4: For each process i from *0* to $n - 1$:

- Initialize identifier: id[i] = i.
- Prompt for and read the burst time into bt[i].

Step 5: Sort processes in ascending order of burst time using selection sort:

- For $i = 0$ to $n - 2$:
  - For $j = i + 1$ to $n - 1$:
    - If bt[j] < bt[i], swap both bt[i] with bt[j] and id[i] with id[j] using a temporary variable x.

Step 6: For each sorted process i from 0 to $n - 1$:

- Set waiting time: wt[i] = t.
- Update cumulative timeline: t = t + bt[i].
- Print process metrics: "P<id[i]>: Waiting=<wt[i]>, Completion=<t>".

Step 7: Stop.

Program

```
#include <stdio.h>

int main(){

    int n,bt[10],id[10],wt[10],t=0;

    printf("Enter number of processes: "); scanf("%d",&n);

    for(int i=0;i<n;i++){ id[i]=i; printf("Enter burst time of P%d: ",i); scanf("%d",&bt[i]); }

    for(int i=0;i<n-1;i++) for(int j=i+1;j<n;j++) if(bt[j]<bt[i]){ int x=bt[i];bt[i]=bt[j];bt[j]=x;
x=id[i];id[i]=id[j];id[j]=x; }

    for(int i=0;i<n;i++){ wt[i]=t; t+=bt[i]; printf("P%d: Waiting=%d,
Completion=%d\n",id[i],wt[i],t); }

    return 0;

}
```

Output

```
meghana@Meghanabook4:~$ gcc FCFS.c -ofile
meghana@Meghanabook4:~$ ./file
Enter number of processes: 3
Enter burst time of P0: 5
Enter burst time of P1: 3
Enter burst time of P2: 8
P0: Waiting=0, Completion=5
P1: Waiting=5, Completion=8
P2: Waiting=8, Completion=16
meghana@Meghanabook4:~$ gcc SJF.c -ofile
meghana@Meghanabook4:~$ ./file
Enter number of processes: 3
Enter burst time of P0: 5
Enter burst time of P1: 3
Enter burst time of P2: 8
P1: Waiting=0, Completion=3
P0: Waiting=3, Completion=8
P2: Waiting=8, Completion=16
```

EXPERIMENT 9

TITLE - Priority and round-robin scheduling.

Aim : To study the concepts of priority and round-robin scheduling

1. Priority Scheduling

Priority Scheduling is a CPU scheduling algorithm that assigns an explicit numerical priority rank to each process in the ready queue, directing the operating system dispatcher to allocate the CPU to the highest-priority runnable process first. It can operate in either preemptive or non-preemptive modes; its primary operational challenge is starvation (indefinite blocking of low-priority tasks), which is conventionally mitigated via aging—gradually increasing the priority of processes that wait in the system over extended durations.

2. Preemptive Mode

Preemptive Mode is an operating system scheduling discipline wherein the kernel's scheduler retains the authority to forcibly suspend a currently executing process prior to its completion or voluntary surrender of the processor. Preemption is triggered by asynchronous system events—such as hardware timer interrupts, higher-priority task arrivals, or I/O completions—allowing the OS to preserve system responsiveness, enforce

strict priority hierarchies, and context-switch the CPU to higher-urgency workloads.

3. Non-Preemptive Mode

Non-Preemptive Mode (also known as Cooperative Scheduling) is an operating system scheduling discipline in which an executing process retains continuous control of the CPU until it voluntarily yields execution, requests an input/output operation, or terminates. Because the scheduler cannot forcibly interrupt a running process mid-burst, context-switching overhead is reduced and race conditions inside the kernel are minimized, but overall system responsiveness is vulnerable to long-running or misbehaved tasks monopolizing the processor.

4. Round-Robin (RR) Scheduling

Round-Robin (RR) Scheduling is a classic, preemptive, time-sharing CPU scheduling algorithm specifically designed for interactive multi-programmed systems, wherein the ready queue is structured as a circular First-In, First-Out (FIFO) buffer and each process is allocated the processor for a fixed, uniform slice of time known as a time quantum or time slice. If a process does not complete within its allotted quantum, a timer interrupt preempts it, its context is preserved, it is placed at the tail of the ready queue, and the CPU is dispatched to the next process.

Domain : Mobile Computing and Embedded Systems

Use case : Smartphone operating system

Priority.

Algorithm.

Step 1: Start.

Step 2: Declare integer variables `n`, loop counters `i` and `j`, a temporary variable `temp`, and arrays `bt[10]` (burst times) and `p[10]` (process identifiers).

Step 3: Prompt for and read the total number of processes `n`.

Step 4: For each process index `i` from `0` to `n - 1`:

- Assign the process identifier: `p[i] = i + 1`.

- Prompt for and read the burst time into `bt[i]`.

Step 5: Sort the processes in ascending order of their burst times using nested selection sort:

- Loop `i` from `0` to `n - 2`:
    - Loop `j` from `i + 1` to `n - 1`:
        - If `bt[i] > bt[j]`:
            - Swap `bt[i]` and `bt[j]` using `temp`.
            - Swap `p[i]` and `p[j]` using `temp`.

Step 6: Print `"Priority Order: "` followed by the sorted process IDs `p[i]` for `i` from `0` to `n - 1`.

Step 7: Stop.

Program.

```
#include <stdio.h>


int main() {
    int n, i, j, bt[10], p[10], temp;


    printf("Enter number of processes: ");
    scanf("%d", &n);


    for(i=0;i<n;i++) {
        p[i]=i+1;
        printf("Enter burst time for P%d: ",p[i]);
        scanf("%d",&bt[i]);
```

```
    }

    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(bt[i]>bt[j]) {
                temp=bt[i]; bt[i]=bt[j]; bt[j]=temp;
                temp=p[i]; p[i]=p[j]; p[j]=temp;
            }

    printf("Priority Order: ");
    for(i=0;i<n;i++)
        printf("P%d ",p[i]);

    return 0;
}
```

Round robin

Algorithm.

Step 1: Start.

Step 2: Declare integer variables `n` (number of processes), `i` (loop index), `q` (time quantum), `t = 0` (elapsed time tracker), `done` (completion flag), and integer arrays `bt[10]` (burst times) and `r[10]` (remaining burst times).

Step 3: Prompt for and read the total number of processes `n`.

Step 4: For each process index `i` from `0` to `n - 1`:

- Prompt for and read the burst time into `bt[i]`.
- Initialize the remaining burst time: `r[i] = bt[i]`.

Step 5: Prompt for and read the time slice value `q`.

Step 6: Print `"Round Robin Order: "`.

Step 7: Execute the scheduling loop indefinitely (`while(1)`):

- Set the termination flag `done = 1`.
- Iterate through all processes from `i = 0` to `n - 1`:
    - If the remaining time `r[i] > 0`:
        - Set `done = 0` (indicating active tasks still remain).
        - Print the dispatched process: `"P<i+1> "`.
        - If `r[i] > q`, decrement remaining burst: `r[i] = r[i] - q`.
        - Otherwise, mark process finished: `r[i] = 0`.
- If `done == 1` (all processes have `r[i] == 0`), break the loop.

Step 8: Stop.

Program.

```
#include <stdio.h>


int main() {
    int n,i,bt[10],r[10],q,t=0,done;


    printf("Enter number of processes: ");
    scanf("%d",&n);


    for(i=0;i<n;i++) {
        printf("Enter burst time for P%d: ",i+1);
```

```
        scanf("%d",&bt[i]);
        r[i]=bt[i];
    }

    printf("Enter time quantum: ");
    scanf("%d",&q);

    printf("Round Robin Order: ");
    while(1) {
        done=1;
        for(i=0;i<n;i++)
            if(r[i]>0) {
                done=0;
                printf("P%d ",i+1);
                if(r[i]>q) r[i]-=q;
                else r[i]=0;
            }
        if(done) break;
    }

    return 0;
}
```

Output. :

```
meghana@Meghanabook4:~$ nano priority.c
meghana@Meghanabook4:~$ gcc priority.c -ofile
meghana@Meghanabook4:~$ ./file
Enter number of processes: 4
Enter burst time for P1: 5
Enter burst time for P2: 2
Enter burst time for P3: 3
Enter burst time for P4: 1
Priority Order: P4 P2 P3 P1 meghana@Meghanabook4:~$ nano roundrobin.c
meghana@Meghanabook4:~$ gcc roundrobin.c -ofile
meghana@Meghanabook4:~$ ./file
Enter number of processes: 4
Enter burst time for P1: 5
Enter burst time for P2: 3
Enter burst time for P3: 8
Enter burst time for P4: 2
Enter time quantum: 2
Round Robin Order: P1 P2 P3 P4 P1 P2 P3 P1 P3 P3 meghana@Meghanabook4:~$
```

Priority and round robin :

Algorithm

Step 1: Start.

Step 2: Declare integer variables n, i, j, t, q, temp, time = 0, and integer arrays bt[10] (burst time), p[10] (process ID), and r[10] (remaining burst time).

Step 3: Prompt for and read the total number of processes n.

Step 4: For each process i from *0* to $n - 1$:

- Initialize process identifier: p[i] = i + 1.
- Read burst time into bt[i].
- Initialize remaining burst time: r[i] = bt[i].

Step 5 (Priority Scheduling Phase):

- Sort arrays bt and p in ascending order of burst time using nested selection sort:
    - For $i = 0$ to $n - 2$:
        - For $j = i + 1$ to $n - 1$:
            - If bt[i] > bt[j], swap bt[i] with bt[j] and p[i] with p[j] using temp.
- Print "Priority Order: " followed by sorted process sequence p[i] for $i = 0$ to $n - 1$.

Step 6 (Round-Robin Preparation):

- Prompt for and read the time slice value q.
- Print "Round Robin Order: ".
- Reinitialize remaining burst array: set r[i] = bt[i] for $i = 0$ to $n - 1$ using the sorted burst values.

Step 7 (Round-Robin Execution Phase):

- Execute infinite loop (while(1)):
  - Set termination flag t = 1.
  - For $i = 0$ to $n - 1$:
    - If r[i] > 0:
      - Set t = 0 (unfinished processes exist).
      - Print "P<p[i]> ".
      - If r[i] > q, decrement: r[i] = r[i] - q.
      - Otherwise, set process to complete: r[i] = 0.
  - If t == 1 (all processes completed), break out of the loop.

Step 8: Stop.

Program.

```
#include <stdio.h>

int main() {
    int n,i,j,t,bt[10],p[10],r[10],q,temp,time=0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++) {
```

```
    p[i]=i+1;
    printf("Enter burst time for P%d: ",p[i]);
    scanf("%d",&bt[i]);
    r[i]=bt[i];
}

/* Priority Scheduling */
for(i=0;i<n-1;i++)
    for(j=i+1;j<n;j++)
        if(bt[i]>bt[j]) {
            temp=bt[i]; bt[i]=bt[j]; bt[j]=temp;
            temp=p[i]; p[i]=p[j]; p[j]=temp;
        }

printf("\nPriority Order: ");
for(i=0;i<n;i++)
    printf("P%d ",p[i]);

/* Round Robin */
printf("\n\nEnter time quantum: ");
scanf("%d",&q);
printf("Round Robin Order: ");
for(i=0;i<n;i++)
```

```
        r[i]=bt[i];


    while(1) {
        t=1;
        for(i=0;i<n;i++)
            if(r[i]>0) {
                t=0;
                printf("P%d ",p[i]);
                if(r[i]>q) r[i]-=q;
                else r[i]=0;
            }
        if(t) break;
    }


    return 0;
}
```

Output.

```
meghana@Meghanabook4:~$ gcc priorityandroundrobin.c -ofile
meghana@Meghanabook4:~$ ./file
Enter number of processes: 4
Enter burst time for P1: 5
Enter burst time for P2: 2
Enter burst time for P3: 3
Enter burst time for P4: 1

Priority Order: P4 P2 P3 P1

Enter time quantum: 2
Round Robin Order: P4 P2 P3 P1 P3 P1 P1 meghana@Meghanabook4:~$
```

EXPERIMENT - 10

TITLE - FIFO Page replacement algorithm

Aim: To implement the FIFO page replacement algorithm

Definition.

1. First-In, First-Out (FIFO)

First-In, First-Out (FIFO) is a foundational queuing and scheduling discipline in operating systems wherein concurrent entities—such as processes, execution threads, memory pages, or I/O request packets—are serviced strictly in the chronological order of their arrival into a waiting structure, managed such that the item enqueued first at the head of the linear queue is the first to be dequeued and allocated the requested system resource.

2. FIFO Waiter (FIFO Waiting Mechanism)

A FIFO Waiter is a concurrency control and thread synchronization mechanism in which blocked threads contending for a busy resource (such as a mutual exclusion lock, counting semaphore, monitor, or condition variable) are placed into a strictly ordered First-In, First-Out wait queue, ensuring that once the resource is surrendered, wake-up signals and access rights are granted exclusively to the thread at the head of the queue, thereby eliminating lock-barging, enforcing bounded waiting, and preventing thread starvation.

Domain: Mobile Computing and Memory Architecture

Use Case: FIFO Page Replacement and Low-Memory Killer (LMK) Cache Eviction in Mobile Operating Systems

Algorithm

Algorithm: First-In, First-Out (FIFO) Page Replacement

Step 1: Start.

Step 2: Declare integer variables f (number of frames), n (number of pages), loop indices i and j, next = 0 (circular pointer to the oldest frame), faults = 0 (page fault counter), hit (flag for page hit), and arrays mem[50] (frame table) and pg[100] (reference string).

Step 3: Prompt for and read the total number of frames f and total pages n.

Step 4: Prompt for and read the sequence of n page numbers into array pg.

Step 5: Initialize all frame slots in memory to empty: set mem[i] = -1 for $i = 0$ to $f - 1$.

Step 6: For each page access i from 0 to $n - 1$:

- Initialize hit indicator: hit = 0.
- Search memory frames: Loop j from 0 to $f - 1$:
    - If mem[j] == pg[i], set hit = 1 and continue.
- If hit == 0 (Page Fault occurs):
    - Replace the oldest frame: set mem[next] = pg[i].
    - Advance the circular replacement pointer: next = (next + 1) % f.
    - Increment fault counter: faults = faults + 1.

Step 7: Print the total number of page faults: "Page faults: <faults>".

Step 8: Stop.

Program

```
#include <stdio.h>

int main() {
```

```
    int f, n, i, j, next = 0, faults = 0, hit, mem[50], pg[100];
    printf("Frames: "); scanf("%d", &f);
    printf("Number of pages: "); scanf("%d", &n);
    printf("Page sequence: "); for (i = 0; i < n; i++) scanf("%d", &pg[i]);
    for (i = 0; i < f; i++) mem[i] = -1;
    for (i = 0; i < n; i++) {
        for (hit = j = 0; j < f; j++) if (mem[j] == pg[i]) hit = 1;
        if (!hit) { mem[next] = pg[i]; next = (next + 1) % f; faults++; }
    }
    printf("Page faults: %d\n", faults);
    return 0;
}
```

Output:

```
meghana@Meghanabook4:~$ nano FIFO.c
meghana@Meghanabook4:~$ gcc FIFO.c-ofile
/usr/bin/x86_64-linux-gnu-ld.bfd: cannot find F
collect2: error: ld returned 1 exit status
meghana@Meghanabook4:~$ gcc FIFO.c -ofile
meghana@Meghanabook4:~$ ./file
Frames: 5
Number of pages: 2
Page sequence: 3

2
Page faults: 2
```

EXPERIMENT - 11

TITLE - LRU and LFU page replacement algorithm

Aim: To implement page replacement using LRU and LFU

Definition.

LRU (Least Recently Used)Definition: LRU evicts the item that has not been accessed for the longest duration of time.Core Principle: Relies on temporal locality—if data was accessed recently, it is highly likely to be accessed again soon.Mechanism: Every time a cache hit or insertion occurs, that item is marked as the most recently used (e.g., moved to the front/head of a doubly linked list). When an eviction must happen, the item at the tail (oldest timestamp/least recently accessed) is removed.Typical Implementation: A Hash Map paired with a Doubly Linked List, providing get and put operations.

2. LFU (Least Frequently Used) Definition: LFU evicts the item that has the lowest total access count (frequency) over time .Core Principle: Relies on frequency of usage—items accessed many times in the past are considered more valuable and likely to be needed again. Mechanism : Every item tracks an access counter initialized to 1. On each subsequent access, its counter increments by 1. When an eviction occurs, the item with the minimum access count is evicted. If there is a tie between multiple items with the same minimum frequency, LRU is typically used as a tiebreaker to evict the least recently accessed among them .Typical Implementation: A Hash Map mapping keys to nodes, and another Hash Map mapping frequencies to Doubly Linked Lists (along with a min frequency tracker), providing get and put operations.

Domain: Mobile Computing and Memory Architecture

Use Case: E-commerce product catalog

LRU

Algorithm: Least Recently Used (LRU) Page Replacement

1. Start and declare variables: frames F, request count n, request array req[], frame array page[], usage tracker used[], faults = 0.

2. Initialize all F frames in page[] to -1 (indicating empty) and used[] elements to 0.

3. Read the inputs: F, n, and the n request IDs into req[].

4. For each request i from 0 to n - 1, perform Steps 5 to 9:

5. Search if req[i] is already present in any frame page[j] (0 <= j < F).

6. If present (Hit): Update its last-used timestamp: used[j] = i + 1.

7. If absent (Page Fault):

a. Increment faults by 1.

b. Find the frame index v having the minimum value in used[].

c. Place req[i] into page[v] and update used[v] = i + 1.

8. Display the current request ID and the IDs present in all allocated frames.

9. End of request loop.

10. Display the total page faults and Stop.

Program :

```
#include <stdio.h>
int main() {
    int n, F, req[100], page[20], used[20] = {0}, faults = 0;
    printf("Enter number of frames: ");   scanf("%d", &F);
    printf("Enter number of requests: "); scanf("%d", &n);
    printf("Enter product IDs: ");        for (int i = 0; i < n; i++) scanf("%d", &req[i]);
    for (int j = 0; j < F; j++) page[j] = -1;          // -1 = empty frame
    for (int i = 0; i < n; i++) {
        int j, hit = -1, v = 0;
        for (j = 0; j < F; j++) if (page[j] == req[i]) hit = j;
        if (hit >= 0) used[hit] = i + 1;               // hit: refresh last-used time
        else {
            for (j = 1; j < F; j++) if (used[j] < used[v]) v = j;  // least recently used slot
            page[v] = req[i]; used[v] = i + 1; faults++;
```

```
        }
        printf("Product %d ->", req[i]);
        for (j = 0; j < F; j++) if (page[j] >= 0) printf(" %d", page[j]);
        printf("\n");
    }
    printf("Page faults: %d\n", faults);
}
```

LFU

Algorithm:

Least Frequently Used (LFU) Page Replacement with FIFO Tie-Breaking

1. Start and declare variables: frames F, request count n, arrays req[], page[], freq[], since[], and faults = 0.

2. Initialize all F frames in page[] to -1 (empty) and both freq[] and since[] arrays to 0.

3. Read inputs: number of frames F, total requests n, and the sequence of request IDs into req[].

4. For each request i from 0 to n - 1, perform Steps 5 to 9:

5. Check if req[i] is already present in any frame page[j] (0 <= j < F).

6. If present (Hit): Increment its access frequency counter: freq[j]++.

7. If absent (Page Fault):

a. Increment faults by 1.

b. Identify victim frame v having the minimum frequency freq[v] (break ties using the smallest arrival time since[v]).

c. Set page[v] = req[i], reset freq[v] = 1, and update arrival time since[v] = i + 1.

8. Print the current request ID and the contents of all allocated frames.

9. End of request loop.

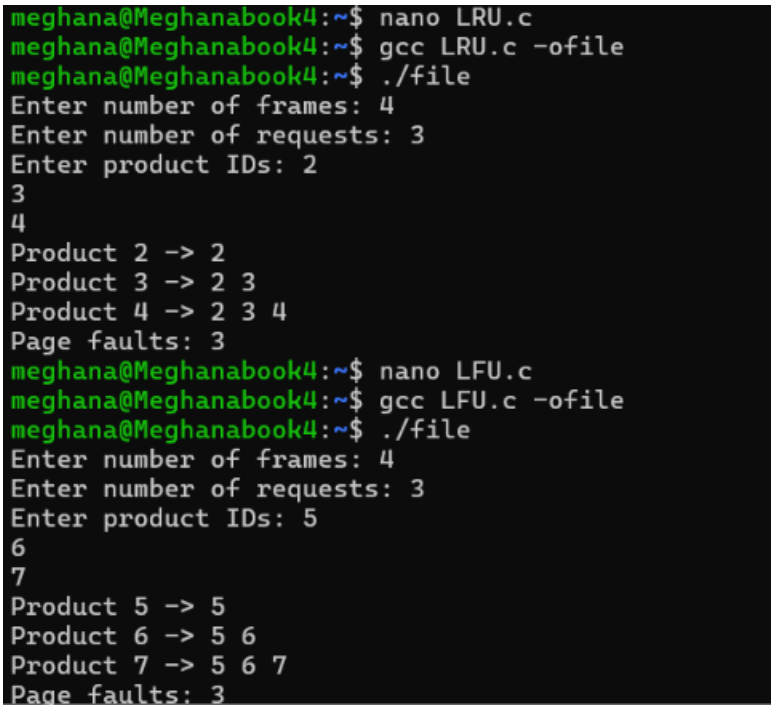
10. Print the total page faults and Stop.

Program :

```
#include <stdio.h>
int main() {
    int n, F, req[100], page[20], freq[20] = {0}, since[20] = {0}, faults = 0;
    printf("Enter number of frames: ");   scanf("%d", &F);
    printf("Enter number of requests: "); scanf("%d", &n);
    printf("Enter product IDs: ");        for (int i = 0; i < n; i++) scanf("%d", &req[i]);
    for (int j = 0; j < F; j++) page[j] = -1;           // -1 = empty frame
    for (int i = 0; i < n; i++) {
        int j, hit = -1, v = 0;
        for (j = 0; j < F; j++) if (page[j] == req[i]) hit = j;
        if (hit >= 0) freq[hit]++;                      // hit: increase access count
        else {
            for (j = 1; j < F; j++)                     // lowest count; oldest wins ties
                if (freq[j] < freq[v] || (freq[j] == freq[v] && since[j] < since[v])) v = j;
            page[v] = req[i]; freq[v] = 1; since[v] = i + 1; faults++;
        }
        printf("Product %d ->", req[i]);
```

```
        for (j = 0; j < F; j++) if (page[j] >= 0) printf(" %d", page[j]);
        printf("\n");
    }
    printf("Page faults: %d\n", faults);
}
```

Output:



Experiment -12

Aim : Best-fit and worst-fit memory management policies

Definition.

Best-Fit Policy

- **Definition:** Allocates the **smallest available free partition** that is large enough to satisfy the requested process size.
- **Mechanism:** The operating system searches the entire list of free partitions (unless the list is kept pre-sorted by size) and chooses the hole that leaves the smallest leftover fragment ($Hole_Size - Process_Size$).
- **Advantage:** Minimizes the immediate leftover space inside the allocated partition, preserving larger blocks intact for subsequent, larger process requests.
- **Disadvantage:** Typically slow because it must scan the entire list to locate the closest match. It also produces very small, fragmented leftover holes across memory (external fragmentation) that are often too tiny to satisfy any future requests.

Worst-Fit Policy

- **Definition:** Allocates the **largest available free partition** to the incoming process.
- **Mechanism:** The operating system scans all available holes (or looks up the head of a list sorted by size in descending order) and assigns the process to the hole with the maximum available capacity.
- **Advantage:** Leaves behind the largest possible residual block after allocation, increasing the likelihood that the leftover space remains large enough to accommodate another process.
- **Disadvantage:** Must examine all partitions (if unsorted) to determine the largest hole. By systematically breaking down large memory blocks, it prevents large processes from finding contiguous space later on.

Domain: Industrial Automation and Smart Manufacturing

Use Case: Dynamic memory allocation

Best fit

Algorithm

1. Start
2. Read no of sizes of memory blocks

3. Read no of sizes of process

4. Larger block,smaller block caculation

5. Allocates the process

6. Reduce remaining size

7. Repeat all process

8. If no suitable block and process as not allocated

9. Stop

PROGRAM

```
#include <stdio.h>


int main() {
    int m, n;


    // Step 2: Read number and sizes of memory blocks
    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    int blockSize[m];
    printf("Enter size of each memory block:\n");
    for (int i = 0; i < m; i++) {
        scanf("%d", &blockSize[i]);
    }


    // Step 3: Read number and sizes of processes
```

```
printf("Enter number of processes: ");
scanf("%d", &n);
int processSize[n];
int allocation[n];
printf("Enter size of each process:\n");
for (int i = 0; i < n; i++) {
    scanf("%d", &processSize[i]);
    allocation[i] = -1; // Initialize allocation as not allocated
}

// Step 7: Repeat for all processes
for (int i = 0; i < n; i++) {
    int bestIdx = -1;

    // Step 4: Search for the smallest suitable block
    for (int j = 0; j < m; j++) {
        if (blockSize[j] >= processSize[i]) {
            if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx]) {
                bestIdx = j;
            }
        }
    }
```

```
        // Step 5 & 6: Allocate and reduce remaining size
        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            blockSize[bestIdx] -= processSize[i];
        }
        // Step 8: If no suitable block, process remains unallocated (-1)
    }

    // Display allocation results
    printf("\nProcess No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);
        if (allocation[i] != -1) {
            printf("%d\n", allocation[i] + 1);
        } else {
            printf("Not Allocated\n");
        }
    }

    // Step 9: Stop
    return 0;
}
```

Output:

```
meghana@Meghanabook4:~$ nano bestfit.c
meghana@Meghanabook4:~$ gcc bestfit.c -ofile
meghana@Meghanabook4:~$ ./file
Enter number of memory blocks: 6
Enter size of each memory block:
1
2
3
4
5
6
Enter number of processes: 5
Enter size of each process:
1
2
3
4
5

Process No.     Process Size    Block No.
1               1               1
2               2               2
3               3               3
4               4               4
5               5               5
```

Worst fit

Algorithm :

Step 1: Start the program.

Step 2: Read the number of memory blocks (n) and input their sizes into array blockSize[].

Step 3: Read the number of processes (m) and input their sizes into array processSize[].

Step 4: Initialize an array allocation[] of size m with -1 to indicate all processes are unallocated.

Step 5: For each process i from 0 to m - 1:

a. Set worstIdx = -1.

b. For each memory block j from 0 to n - 1:

i. If blockSize[j] >= processSize[i]:

- If worstIdx == -1 or blockSize[j] > blockSize[worstIdx], then set worstIdx = j.

c. If worstIdx != -1:

i. Set allocation[i] = worstIdx.

ii. Reduce the available block size: blockSize[worstIdx] = blockSize[worstIdx] - processSize[i].

Step 6: Display the allocation table showing Process Number, Process Size, and Allocated Block Number (or "Not Allocated" if allocation[i] == -1).

Step 7: Stop the program.

Program :

```
#include <stdio.h>

int main() {
    int n, m, i, j;
    int blockSize[20], processSize[20], allocation[20];

    printf("Enter number of memory blocks: ");
    scanf("%d", &n);

    printf("Enter sizes of %d blocks:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &blockSize[i]);
    }

    printf("Enter number of processes: ");
    scanf("%d", &m);

    printf("Enter sizes of %d processes:\n", m);
    for (i = 0; i < m; i++) {
        scanf("%d", &processSize[i]);
        allocation[i] = -1;
    }

    // Worst Fit Algorithm
    for (i = 0; i < m; i++) {
        int worstIdx = -1;
        for (j = 0; j < n; j++) {
            if (blockSize[j] >= processSize[i]) {
```

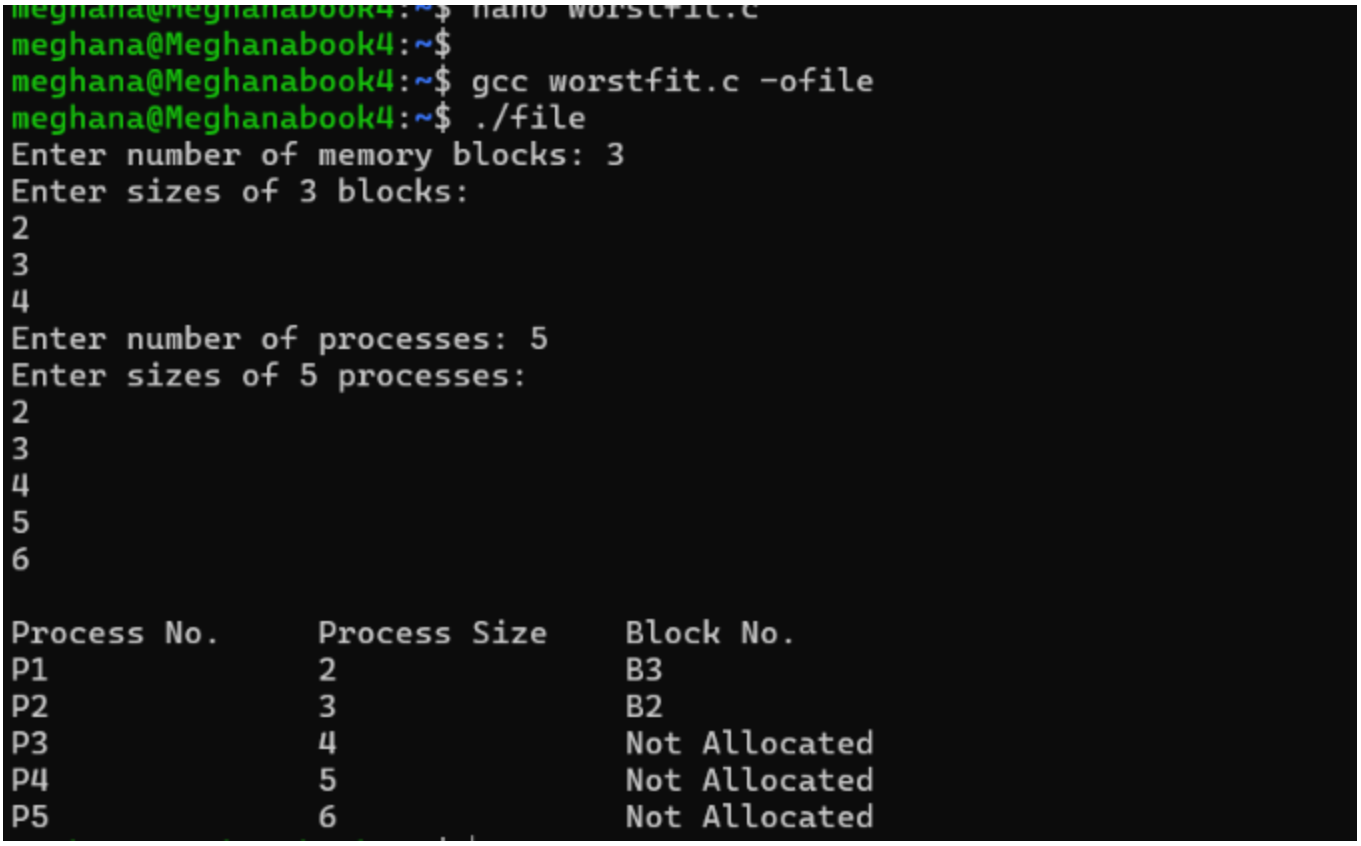
```
            if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx]) {
                worstIdx = j;
            }
        }
    }

    if (worstIdx != -1) {
        allocation[i] = worstIdx;
        blockSize[worstIdx] -= processSize[i];
    }
}

// Output Table
printf("\nProcess No.\tProcess Size\tBlock No.\n");
for (i = 0; i < m; i++) {
    printf("P%d\t\t%d\t\t", i + 1, processSize[i]);
    if (allocation[i] != -1) {
        printf("B%d\n", allocation[i] + 1);
    } else {
        printf("Not Allocated\n");
    }
}

return 0;
}
```

Output:



COMBINATION :

Algorithm :

Step 1: Start.

Step 2: Initialize arrays for block sizes b[] and process sizes p[].

Step 3: Define a function fit(mode):

a. Make a fresh copy B[] of the original block sizes b[].

b. For each process i:

i. Set index k = -1.

ii. For each block j:

- If B[j] >= p[i] and (k == -1 or condition matches):

* Condition: B[j] < B[k] (for Best Fit) OR B[j] > B[k] (for Worst Fit).

* Update k = j.

iii. If k != -1, allocate block (print B[k+1]) and reduce capacity: B[k] = B[k] - p[i].
Else, print "NA" (Not Allocated).

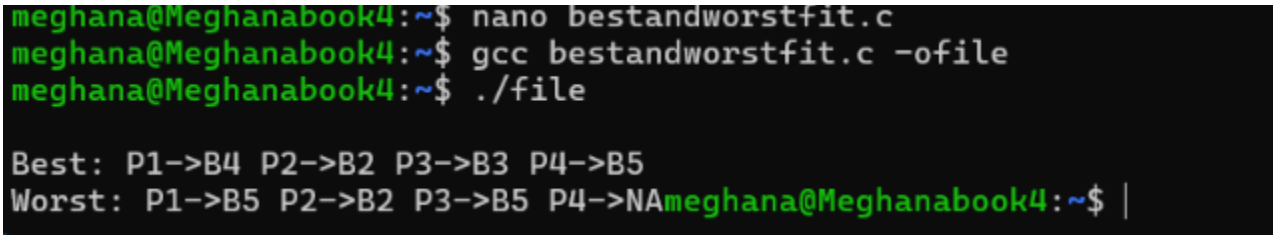
Step 4: Execute fit() for Best Fit, then execute fit() for Worst Fit.

Step 5: Stop.

Program :

```
#include <stdio.h>
int b[]={100,500,200,300,600}, p[]={212,417,112,426}, n=5, m=4;
void fit(char *s, int best) {
   int B[5], i, j, k; for(i=0; i<n; i++) B[i]=b[i];
   printf("\n%s:", s);
   for(i=0; i<m; i++) {
      for(k=-1, j=0; j<n; j++) if(B[j]>=p[i] && (k==-1 || (best ? B[j]<B[k] : B[j]>B[k]))) k=j;
      if(k!=-1) { printf(" P%d->B%d", i+1, k+1); B[k]-=p[i]; } else printf(" P%d->NA", i+1);
   }
}
int main() { fit("Best", 1); fit("Worst", 0); return 0; }
```

Output :



Experiment No. 13: Disk Scheduling (SSTF)

Aim

To study and implement the Shortest Seek Time First (SSTF) disk scheduling algorithm.

Definitions

- **Seek Time**: The duration required for the read/write head of the disk arm to position itself over the specified data cylinder track.
-
- **SSTF (Shortest Seek Time First)**: A disk scheduling algorithm that services the pending track request closest to the current read/write head position to minimize head movement.
-
- **Total Head Movement**: The total cumulative tracks traversed across all serviced requests.
-

Use Case

E-Commerce: Speeds up database queries during flash sales by fetching cart and product data stored on adjacent disk tracks.

Algorithm

1. Start the program.
2.
3. Read the total number of track requests (n) and the current initial head position.
4.
5. Read the array of requested cylinder tracks.
6.
7. Initialize `total_movement = 0` and mark all requests as unvisited (`visited[j] = 0`).
8.
9. Repeat for all n requests:
10.
    - Find the unvisited track index that has the minimum absolute difference from the current head position.
    -
    - Mark that track as visited.
    -
    - Add the absolute distance to `total_movement`.
    -
    - Update the current head position to this chosen track.
    -
11. Display the complete sequence of serviced tracks and the total head movement.
12.
13. Stop the program.
14.

Program

C

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main() {
    int n, head, total_movement = 0;
    int req[50], visited[50] = {0};

    printf("Enter number of requests: ");
    if (scanf("%d", &n) != 1 || n <= 0) return 1;

    printf("Enter disk track requests:\n");
    for (int i = 0; i < n; i++) {
```

```
        printf(" Request %d: ", i + 1);
        scanf("%d", &req[i]);
    }

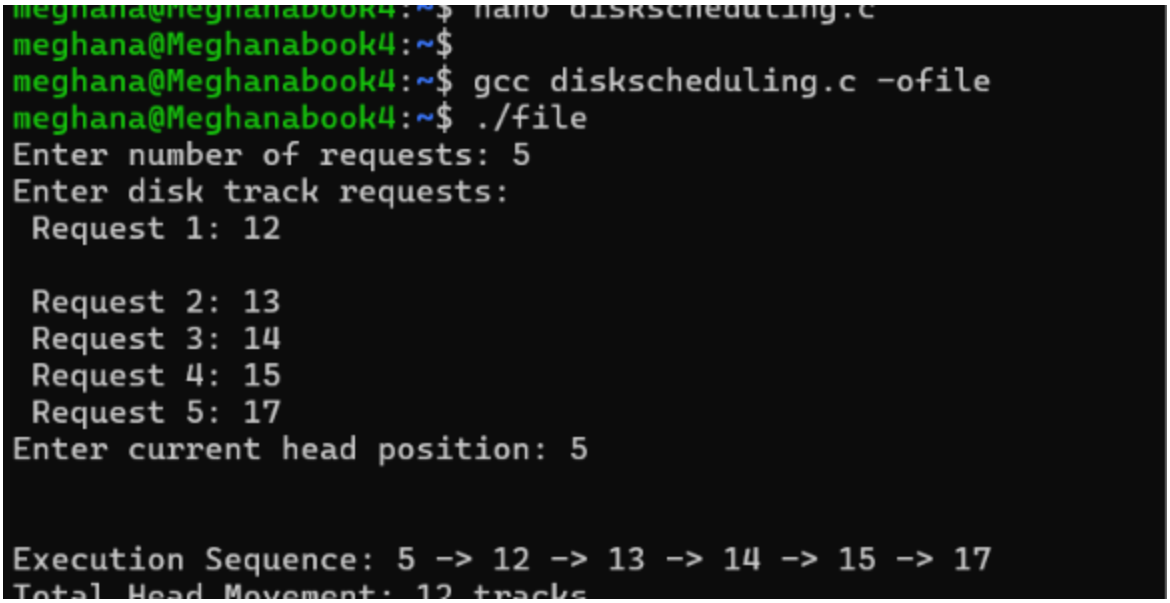
    printf("Enter current head position: ");
    scanf("%d", &head);

    printf("\nExecution Sequence: %d", head);

    for (int i = 0; i < n; i++) {
        int min_dist = 1e9, next_idx = -1;
        for (int j = 0; j < n; j++) {
            if (!visited[j] && abs(req[j] - head) < min_dist) {
                min_dist = abs(req[j] - head);
                next_idx = j;
            }
        }
        visited[next_idx] = 1;
        total_movement += min_dist;
        head = req[next_idx];
        printf(" -> %d", head);
    }

    printf("\nTotal Head Movement: %d tracks\n", total_movement);
    return 0;
}
```

Output :



Experiment No. 14: SCAN (Elevator) Disk Scheduling

Aim

To study and implement the SCAN (Elevator) disk scheduling algorithm.

Definitions

- **SCAN Algorithm**: An algorithm where the disk arm moves in one direction servicing requests until it reaches the edge track, then reverses direction to service requests along the return path.
-
- **Elevator Principle**: Mimics an elevator by continuing in its current direction until the top or bottom floor is reached before turning back.
-
- **Starvation Prevention**: Reduces the waiting time variance compared to SSTF by guaranteeing a sweep across the whole disk surface.
-

Use Case

Operating System File Servers: Handles batched read/write traffic evenly without heavily penalizing requests on the edges of the disk.

Algorithm:

Contiguous (Sequential) File Allocation Simulation

1. Start and declare disk array of size 50 initialized to 0 (0 = free, 1 = allocated), variables start, len, i, ok, and more = 1.

2. While more is equal to 1, execute Steps 3 to 9:
3. Read the starting block index start and length of file len from the user.
4. Verify boundary limits: check if start >= 0 and start + len <= 50; set flag ok = 1 if within range, else ok = 0.
5. Check availability: for each block i from start to start + len - 1, if disk[i] is already allocated (1), set ok = 0 and break.
6. If ok == 1 (Allocation Success):
   a. Iterate i from start to start + len - 1.
   b. Mark each block as occupied: disk[i] = 1.
   c. Print the allocated block numbers.
7. If ok == 0 (Allocation Failure):
   a. Display "File not allocated (blocks busy or out of range)".
8. Prompt user "Store another video? (1/0)" and read input into more.
9. End of while loop.
10. Stop.

Program:

```
#include <stdio.h>
int main() {
   int disk[50] = {0}, start, len, i, ok, more = 1;     // 0 = free block, 1 = used block
   while (more) {
      printf("Enter start block and length of video file: "); scanf("%d %d", &start, &len);
      ok = (start >= 0 && start + len <= 50);
      for (i = start; ok && i < start + len; i++) if (disk[i]) ok = 0;   // all blocks must be free
      if (ok) { printf("Blocks allocated:"); for (i = start; i < start + len; i++) { disk[i] = 1; printf(" %d", i); } printf("\n"); }
      else printf("File not allocated (blocks busy or out of range)\n");
      printf("Store another video? (1/0): "); scanf("%d", &more);
   }
}
```

Algorithm:

Indexed File Allocation Simulation

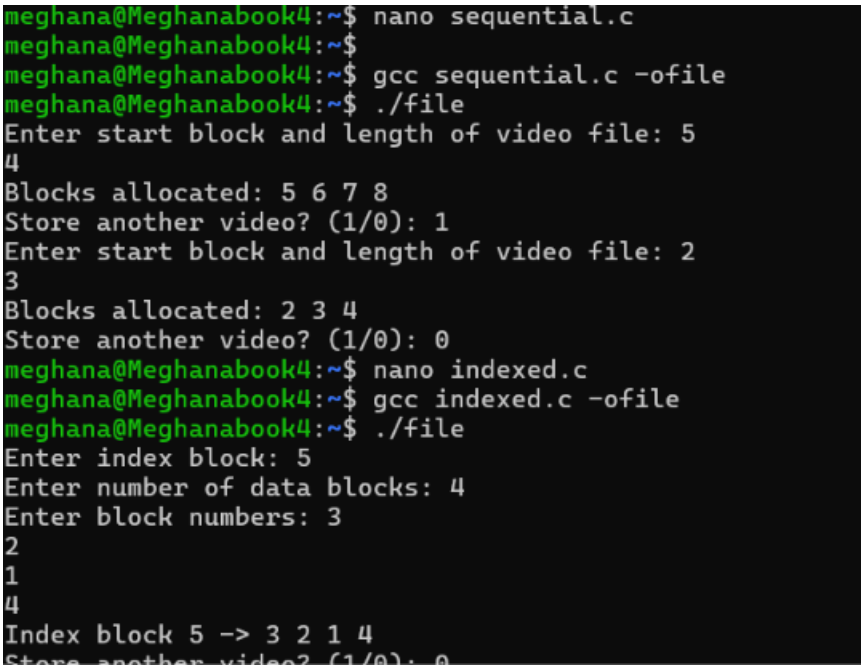
1. Start and initialize disk array of size 50 with all elements set to 0 (0 = free, 1 = allocated), and set more = 1.
2. While more is equal to 1, repeat Steps 3 to 9:
3. Read user inputs: index block number idx, total data blocks n, and the n data block numbers into array blk[].
4. Validate index block: check if idx is within range [0, 49] and disk[idx] == 0; set status flag ok = 1 if valid, else ok = 0.
5. Validate data blocks: for each block blk[i], check if it is within range [0, 49], free (disk[blk[i]] == 0), and not equal to idx. If any check fails, set ok = 0.
6. If ok == 1 (Valid Allocation):
   a. Mark index block as allocated: disk[idx] = 1.
   b. For each data block i from 0 to n - 1, mark disk[blk[i]] = 1 and display blk[i].
   c. Print the allocated index block and its linked data blocks.
7. If ok == 0 (Allocation Failed):
   a. Display "File not allocated (blocks busy or out of range)".
8. Prompt user to continue: "Store another video? (1/0)" and read input into variable more.
9. End of while loop.
10. Stop.

Program:

```
#include <stdio.h>
int main() {
   int disk[50] = {0}, idx, n, blk[50], i, ok, more = 1;   // 0 = free block, 1 = used block
   while (more) {
      printf("Enter index block: ");           scanf("%d", &idx);
      printf("Enter number of data blocks: ");  scanf("%d", &n);
      printf("Enter block numbers: ");         for (i = 0; i < n; i++) scanf("%d", &blk[i]);
      ok = (idx >= 0 && idx < 50 && !disk[idx]);
      for (i = 0; i < n; i++) if (blk[i] < 0 || blk[i] >= 50 || disk[blk[i]] || blk[i] == idx) ok = 0;
```

```
        if (ok) { disk[idx] = 1; printf("Index block %d ->", idx); for (i = 0; i < n; i++) {
disk[blk[i]] = 1; printf(" %d", blk[i]); } printf("\n"); }
        else printf("File not allocated (blocks busy or out of range)\n");
        printf("Store another video? (1/0): "); scanf("%d", &more);
    }
}
```

Output :



Experiment No. 15: File Allocation Strategy (Indexed Allocation)

Aim

To simulate and implement the Indexed File Allocation strategy used in file system storage.

Definitions

- **File Allocation Table**: A data structure kept by the OS to map logical file addresses to physical disk sectors.
-
- **Indexed Allocation**: A file allocatiknon method where each file has its own index block storing the disk pointers to all its scattered data blocks.
-

- **Direct Access**: The ability to access an arbitrary block directly using its index pointer without traversing prior blocks.
-

Use Case

Database Storage: Efficiently maps scattered, non-contiguous disk sectors to a single file without external fragmentation.

Algorithm:

Single-Level Directory File Management Simulation

1. Start and declare a 2D character array files[50][20], string name, and integer variables n = 0, ch, i, found.
2. Begin an infinite loop to display the menu (1. Save, 2. Delete, 3. Search, 4. Display, 5. Exit) and read choice ch.
3. If ch == 5, terminate the loop and jump to Step 10.
4. If ch is between 1 and 3, prompt for and read the target file name.
5. Search files: set found = -1, iterate i from 0 to n - 1, and if files[i] equals name, set found = i.
6. Case ch == 1 (Save):
   a. If found >= 0, display "Name already exists in root directory".
   b. Else if n == 50, display "Directory full".
   c. Else, copy name into files[n], increment n by 1, and display "Saved".
7. Case ch == 2 (Delete):
   a. If found < 0, display "Photo not found".
   b. Else, shift subsequent elements from found to n - 2 one position left, decrement n by 1, and display "Deleted".
8. Case ch == 3 (Search): Display "Found" if found >= 0, otherwise display "Not found".
9. Case ch == 4 (Display): Print all active filenames stored from index 0 to n - 1.
10. Stop.

Program

```
#include <stdio.h>
#include <string.h>
int main() {
   char files[50][20], name[20] = "";
   int n = 0, ch, i, found;
   while (1) {
      printf("\n1.Save photo 2.Delete 3.Search 4.Display 5.Exit: "); scanf("%d",
&ch);
```

```
        if (ch == 5) break;
        if (ch >= 1 && ch <= 3) { printf("Enter photo name: "); scanf("%19s", name); }
        for (found = -1, i = 0; i < n; i++) if (!strcmp(files[i], name)) found = i;
        if (ch == 1) {
            if (found >= 0) printf("Name already exists in root directory\n");
            else if (n == 50) printf("Directory full\n");
            else { strcpy(files[n++], name); printf("Saved\n"); }
        } else if (ch == 2) {
            if (found < 0) printf("Photo not found\n");
            else { for (i = found; i < n - 1; i++) strcpy(files[i], files[i+1]); n--;
printf("Deleted\n"); }
        } else if (ch == 3) printf("%s\n", found >= 0 ? "Found" : "Not found");
        else if (ch == 4) { printf("Root:"); for (i = 0; i < n; i++) printf(" %s", files[i]);
printf("\n"); }
    }
}
```

Algorithm:

Two-Level Directory Management Simulation

1. Start and declare arrays dir[10][20], files[10][20][20], strings d, f, counters cnt[10] = {0}, nd = 0, and loop variables.
2. Begin an infinite loop to display the menu (1.Create folder, 2.Save photo, 3.Delete photo, 4.Search photo, 5.Display all, 6.Exit) and read choice ch.
3. If ch == 6, break the loop and proceed to Step 10.
4. If ch is between 1 and 4, read target folder name d and search dir[] to set folder index di (di = -1 if absent).
5. Case ch == 1 (Create folder): If di >= 0, display "Folder already exists"; else if nd == 10, display "Directory full"; else add d to dir[nd++], display "Folder created".
6. Case ch == 5 (Display all): Iterate through all active directories and print each folder name along with all its saved file names.
7. For operations ch in [2, 4]: If di < 0, display "Folder not found" and continue to Step 2; otherwise read file name f and search files[di][] to find file index fi.

8. Case ch == 2 (Save photo): If fi >= 0, display "Photo already in this folder"; else if cnt[di] == 20, display "Folder full"; else copy f to files[di][cnt[di]++], display "Saved".
9. Case ch == 3 (Delete photo) and Case ch == 4 (Search photo):
   a. If ch == 3: If fi < 0, display "Photo not found"; else shift subsequent files left in files[di], decrement cnt[di], display "Deleted".
   b. If ch == 4: Display "Found" if fi >= 0, else display "Not found".
10. Stop.

Program:

```
#include <stdio.h>
#include <string.h>
int main() {
   char dir[10][20], files[10][20][20], d[20] = "", f[20] = "";
   int cnt[10] = {0}, nd = 0, ch, i, j, di, fi;
   while (1) {
      printf("\n1.Create folder 2.Save photo 3.Delete photo 4.Search photo 5.Display all 6.Exit: "); scanf("%d", &ch);
      if (ch == 6) break;
      if (ch >= 1 && ch <= 4) { printf("Enter folder name: "); scanf("%19s", d); }
      for (di = -1, i = 0; i < nd; i++) if (!strcmp(dir[i], d)) di = i;
      if (ch == 1) {
         if (di >= 0) printf("Folder already exists\n");
         else if (nd == 10) printf("Directory full\n");
         else { strcpy(dir[nd++], d); printf("Folder created\n"); }
      } else if (ch == 5) {
         for (i = 0; i < nd; i++) { printf("%s:", dir[i]); for (j = 0; j < cnt[i]; j++) printf(" %s", files[i][j]); printf("\n"); }
      } else if (ch >= 2 && ch <= 4) {
         if (di < 0) { printf("Folder not found\n"); continue; }
         printf("Enter photo name: "); scanf("%19s", f);
         for (fi = -1, j = 0; j < cnt[di]; j++) if (!strcmp(files[di][j], f)) fi = j;
         if (ch == 2) {
```

```
            if (fi >= 0) printf("Photo already in this folder\n");
            else if (cnt[di] == 20) printf("Folder full\n");
            else { strcpy(files[di][cnt[di]++], f); printf("Saved\n"); }
        } else if (ch == 3) {
            if (fi < 0) printf("Photo not found\n");
            else { for (j = fi; j < cnt[di] - 1; j++) strcpy(files[di][j], files[di][j+1]);
cnt[di]--; printf("Deleted\n"); }
        } else printf("%s\n", fi >= 0 ? "Found" : "Not found");
    }
  }
}
```

Output:

```
meghana@Meghanabook4:~$ gcc singledirectory.c -ofile
meghana@Meghanabook4:~$ ./file

1.Save photo 2.Delete 3.Search 4.Display 5.Exit: 3
Enter photo name: picture
Not found

1.Save photo 2.Delete 3.Search 4.Display 5.Exit: 1
Enter photo name: 123
Saved

1.Save photo 2.Delete 3.Search 4.Display 5.Exit: 3
Enter photo name: 123
Found

1.Save photo 2.Delete 3.Search 4.Display 5.Exit: 5
meghana@Meghanabook4:~$ nano twodirectory.c
meghana@Meghanabook4:~$ nano twodirectory.c
meghana@Meghanabook4:~$ gcc twodirectory.c -ofile
meghana@Meghanabook4:~$ ./file

1.Create folder 2.Save photo 3.Delete photo 4.Search photo 5.Display all 6.Exit: 1
Enter folder name: hello
Folder created

1.Create folder 2.Save photo 3.Delete photo 4.Search photo 5.Display all 6.Exit: 5
hello:

1.Create folder 2.Save photo 3.Delete photo 4.Search photo 5.Display all 6.Exit: 6
```